SETUP GUIDE

APACHE AIRFLOW

Companion Document for Hands-On Topic 4

Purpose:

- This guide walks you through installing Apache Airflow on your local machine.
- Follow the section that matches your operating system.
- At the end, you will verify the installation by running a test DAG.
- Complete this setup BEFORE the live session so you arrive ready to build real pipelines.

PREREQUISITES (ALL PLATFORMS):

- Python 3.10 or 3.11 (recommended). Minimum 3.8.
- pip (comes with Python)
- Git (optional, useful for cloning example repos)
- At least 4 GB free RAM
- Basic comfort with a terminal / command line

SECTION A: WINDOWS (WSL) — RECOMMENDED FOR WINDOWS USERS

WSL (Windows Subsystem for Linux) gives you a real Linux terminal inside Windows.
Airflow runs natively on Linux, so WSL is the cleanest path.

A1. INSTALL WSL (SKIP IF ALREADY INSTALLED)

Open PowerShell as Administrator and run:
wsl –install

This installs Ubuntu by default. Restart your computer when prompted.
After restart, open "Ubuntu" from the Start menu.
It will ask you to create a username and password — remember these.

Verify:

```
wsl --version
```

You should see WSL version 2.x and Ubuntu listed.

A2. INSTALL PYTHON INSIDE WSL

Ubuntu comes with Python, but let us make sure it is the right version:

```
sudo apt update && sudo apt upgrade -y

sudo apt install python3 python3-pip python3-venv -y

python3 --version
```

You should see Python 3.10.x or 3.11.x (depending on your Ubuntu version).

If you see 3.8, upgrade:

```
sudo apt install software-properties-common -y

sudo add-apt-repository ppa:deadsnakes/ppa -y

sudo apt update

sudo apt install python3.11 python3.11-venv python3.11-dev -y
```

A3. CREATE VIRTUAL ENVIRONMENT & INSTALL AIRFLOW

```
python3 -m venv ~/airflow_env

source ~/airflow_env/bin/activate
```

Your prompt should now show (airflow_env). Set the Airflow home directory:

export AIRFLOW_HOME=~/airflow

Install Airflow with the constraints file (prevents dependency conflicts):

```
pip install "apache-airflow==2.8.1" --constraint \
```

"https://raw.githubusercontent.com/apache/airflow/constraints-2.8.1/constraints-3.10.txt"

NOTE: Match the constraints file to YOUR Python version:

Python 3.10 → constraints-3.10.txt
Python 3.11 → constraints-3.11.txt

IMPORTANT — PYTHON 3.12+ USERS:

Airflow 2.8.1 does NOT have a constraints file for Python 3.12 or later.

If you run "python3 --version" and see 3.12.x, use Airflow 2.10.4 instead:

```
pip install "apache-airflow==2.10.4" --constraint \
```

"https://raw.githubusercontent.com/apache/airflow/constraints-2.10.4/constraints-3.12.txt"

All subsequent steps (db init, user creation, webserver, scheduler) remain the same regardless of which Airflow version you installed.

This takes 3–5 minutes depending on your internet speed.

A4. INITIALISE AIRFLOW

airflow db init

This creates:

~/airflow/airflow.cfg (configuration file)

~/airflow/airflow.db (SQLite metadata database)

Now create the DAGs folder (some Airflow versions do not create it automatically):

```
mkdir -p ~/airflow/dags
```

Disable example DAGs (they clutter the UI and slow down the scheduler):

sed -i 's/^load_examples = True/load_examples = False/' ~/airflow/airflow.cfg

Create an admin user:

airflow users create \

--username admin \

--password admin \

--firstname Admin \

--lastname User \

--role Admin \

--email admin@example.com

A5. START AIRFLOW

You need TWO terminal windows (both inside WSL, both with the venv active).

Terminal 1 — Web Server:

```
source ~/airflow_env/bin/activate
```

export AIRFLOW_HOME=~/airflow

airflow webserver --port 8080

Terminal 2 — Scheduler:

source ~/airflow_env/bin/activate

export AIRFLOW_HOME=~/airflow
airflow scheduler
Open your browser (in Windows) and go to:
http://localhost:8080

Log in with username: admin, password: admin.
You should see the Airflow dashboard.

A6. VERIFY WITH A TEST DAG

Create the test DAG file:

nano ~/airflow/dags/test_dag.py

Paste the code below. IMPORTANT: copy starting from the "f" in "from" —

do NOT include any leading spaces before the first column:

——————————— start of test_dag.py ———————————

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime

def hello():

print("Airflow is working!")

with DAG(

'test_dag',

start_date=datetime(2024, 1, 1),

schedule_interval=None,

catchup=False

) as dag:

PythonOperator(

task_id='hello_task',
```

```
    python_callable=hello
)
```

——————————— end of test_dag.py ———————————

Save (Ctrl+O, Enter, Ctrl+X in nano).

VERIFY BEFORE MOVING ON — run this command:

```
python3 ~/airflow/dags/test_dag.py
```

If you see no output, the file is correct. If you see "IndentationError",

the file has extra leading spaces. Fix with: sed -i 's/^ //' ~/airflow/dags/test_dag.py

In the Airflow UI:

1. Wait ~30 seconds for the scheduler to detect the new file
2. Find "test_dag" in the DAG list
3. Toggle the DAG ON (unpause it)
4. Click the play button → "Trigger DAG"
5. Click on the DAG run → click "hello_task" → click "Log"
6. You should see: "Airflow is working!"

If you see that message — your setup is complete.

A7. OPTIONAL: MAKE AIRFLOW START EASIER

Add these lines to your ~/.bashrc so you do not have to type them every time:

```
echo 'alias airflow-start="source ~/airflow_env/bin/activate && export AIRFLOW_HOME=~/airflow"' >> ~/.bashrc
```

```
source ~/.bashrc
```

Now just type "airflow-start" to activate the environment.

SECTION B: macOS

B1. INSTALL PYTHON (IF NEEDED)

macOS comes with Python, but the system version may be old. Use Homebrew:

```
# Install Homebrew if you do not have it:
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

```
brew install python@3.11
```

```
python3 --version
```

Should show Python 3.11.x.

B2. CREATE VIRTUAL ENVIRONMENT & INSTALL AIRFLOW

```
python3 -m venv ~/airflow_env

source ~/airflow_env/bin/activate

export AIRFLOW_HOME=~/airflow

pip install "apache-airflow==2.8.1" --constraint \

"https://raw.githubusercontent.com/apache/airflow/constraints-2.8.1/constraints-3.11.txt"
```

NOTE: Match the constraints file to YOUR Python version.

B3. INITIALISE AIRFLOW

```
airflow db init

airflow users create \

--username admin \

--password admin \

--firstname Admin \

--lastname User \

--role Admin \

--email admin@example.com
```

B4. START AIRFLOW

Terminal 1:

```
source ~/airflow_env/bin/activate

export AIRFLOW_HOME=~/airflow

airflow webserver --port 8080
```

Terminal 2:

```
source ~/airflow_env/bin/activate

export AIRFLOW_HOME=~/airflow
```

airflow scheduler

Open http://localhost:8080 in your browser. Log in with admin / admin.

B5. VERIFY WITH A TEST DAG

nano ~/airflow/dags/test_dag.py
Paste the same test DAG code from Section A6 above. Save.

Follow the same verification steps:
Unpause → Trigger → Check logs → See "Airflow is working!"

SECTION C: LINUX (UBUNTU / DEBIAN)

C1. INSTALL PYTHON

sudo apt update && sudo apt upgrade -y

sudo apt install python3 python3-pip python3-venv -y

python3 --version

C2. CREATE VIRTUAL ENVIRONMENT & INSTALL AIRFLOW

python3 -m venv ~/airflow_env

source ~/airflow_env/bin/activate

export AIRFLOW_HOME=~/airflow

pip install "apache-airflow==2.8.1" --constraint \

"https://raw.githubusercontent.com/apache/airflow/constraints-2.8.1/constraints-3.10.txt"

C3. INITIALISE, STARTAND VERIFY

Same as Sections A4, A5, and A6 — the commands are identical to WSL.

SECTION D: DOCKER COMPOSE (ANY OS — PRODUCTION-LIKE SETUP)

This method gives you a full production-like Airflow setup with PostgreSQL,
Celery workers, and Redis. Requires Docker (covered in Week 4, but if you already have it installed,

this is the best option).

D1. INSTALL DOCKER

Windows: Download Docker Desktop from https://www.docker.com/products/docker-desktop/
→ Enable WSL 2 backend during installation

macOS: Download Docker Desktop (same URL)
Linux: sudo apt install docker.io docker-compose-v2 -y

Verify:
docker --version
docker compose version

D2. DOWNLOAD THE OFFICIAL COMPOSE FILE

mkdir ~/airflow-docker && cd ~/airflow-docker

curl -LfO 'https://airflow.apache.org/docs/apache-airflow/2.8.1/docker-compose.yaml'

Create required directories:

```
mkdir -p ./dags ./logs ./plugins ./config
```

echo -e "AIRFLOW_UID=$(id -u)" > .env

D3. START AIRFLOW

docker compose up airflow-init # First-time initialization

docker compose up -d # Start all services

Wait 1–2 minutes for services to start. Then open:
http://localhost:8080
Default login: airflow / airflow (different from pip method).

D4. VERIFY

Create the test DAG:
nano ~/airflow-docker/dags/test_dag.py
Paste the same test DAG code from Section A6. Save.
Follow the same UI verification steps.

D5. STOP / RESTART

```
docker compose down  # Stop and remove containers

docker compose up -d  # Start again
```

KEY CONFIGURATION CHANGES (ALL METHODS)

After installation, edit airflow.cfg (pip) or docker-compose.yaml (Docker) to apply these settings:

1. DISABLE EXAMPLE DAGS (reduces UI clutter):

In airflow.cfg: load_examples = False

In Docker Compose: AIRFLOW__CORE__LOAD_EXAMPLES: 'false'

2. SWITCH EXECUTOR (pip method only — Docker already uses Celery):

In airflow.cfg: executor = LocalExecutor

NOTE: LocalExecutor requires PostgreSQL, not SQLite.

Quick fix for learning: keep SequentialExecutor with SQLite.

3. SET YOUR DAG FOLDER:

In airflow.cfg: dags_folder = /path/to/your/dags

After changing config, restart both webserver and scheduler.

TROUBLESHOOTING — COMMON ISSUES & FIXES

ISSUE: "ModuleNotFoundError: No module named 'airflow'"

CAUSE: Virtual environment not activated.

FIX: source ~/airflow_env/bin/activate

ISSUE: "Cannot use sqlite with the LocalExecutor"

CAUSE: SQLite does not support concurrent writes.

FIX: Either switch back to SequentialExecutor (for learning) OR

install PostgreSQL and update sql_alchemy_conn in airflow.cfg.

ISSUE: "The scheduler is not running" / Tasks stuck in "scheduled"

CAUSE: Scheduler process not started.

FIX: Open a second terminal, activate venv, run: airflow scheduler

ISSUE: "DAG is not appearing in the UI"

CAUSE: Python syntax error in DAG file.

FIX: Run: python ~/airflow/dags/your_dag.py

Python will show you the exact error and line number.

ISSUE: Airflow webserver shows "502 Bad Gateway"

CAUSE: Webserver did not finish starting.

FIX: Wait 30–60 seconds. If persists, check port 8080 is not in use:

lsof -i :8080

ISSUE: (WSL) "localhost:8080 not loading in Windows browser"

CAUSE: WSL networking not forwarding.
FIX: Try http://127.0.0.1:8080 instead, or check WSL version:
wsl --version (should be WSL 2).

ISSUE: (Docker) Containers keep restarting
CAUSE: Not enough memory allocated to Docker.
FIX: Docker Desktop → Settings → Resources → Memory → set to 4+ GB.

ISSUE: pip install fails with dependency conflicts
CAUSE: Not using the constraints file, or wrong Python version.
FIX: Always use --constraint with the URL matching YOUR Python version.
Check: python3 --version, then use the matching constraints file.

VERIFICATION CHECKLIST
Before the live session, confirm ALL of these:

- [ ] Python 3.10+ installed and working
- [ ] Virtual environment created and activates correctly
- [ ] Airflow webserver starts on http://localhost:8080
- [ ] Airflow scheduler starts without errors
- [ ] You can log in to the UI (admin/admin or airflow/airflow)
- [ ] test_dag appears in the DAG list
- [ ] test_dag runs successfully when triggered
- [ ] Task log shows"Airflow is working!"

If all boxes are checked, you are ready for the live sessions.

QUICK REFERENCE — DAILY COMMANDS

START YOUR SESSION:

```
source ~/airflow_env/bin/activate
export AIRFLOW_HOME=~/airflow
airflow webserver --port 8080    # Terminal 1
irflow scheduler                 # Terminal 2
```

OR (Docker):

```
cd ~/airflow-docker
docker compose up -d
```

STOP:

Ctrl+C in both terminals (pip method)
docker compose down (Docker method)

CHECK RUNNING PROCESSES:

ps aux | grep airflow

CLEAR A STUCK DAG RUN:
airflow dags backfill --reset-dagruns -s 2024-01-01 -e 2024-01-01 test_dag